Ответы на вопросы собеседования

Вакансия:

https://voronezh.hh.ru/vacancy/131850689?hhtmFrom=chat

Список вопросов:

https://docs.google.com/document/d/1oyrBHZoXt78VVsqd3vLDvxDpObKXIapYFCIpE29CUfo/edit?pli=1&tab=t.0

Вопрос 1

Вопрос связан с архитектурой построения таблиц БД и обвязки в виде Кеша. В ответе указать не просто схему организации, но объяснить ход мышления.
Дано: есть бизнес структура. Она состоит из товаров которые продаются в разных ГЕО. При этом в каждом ГЕО есть своя валюта и своя цена (учитывая доставку). К каждому ГЕО, на каждый товар приходят лиды, которых надо фиксировать. При этом есть коэффициент который динамически повышает или понижает цену в зависимости от количества лидов пришедших на ГЕО и товар. По принципу - больше лидов - меньше цена. Рейтинг обновляется раз в 10 минут.
Отразите схему организации мест хранения данных и взаимодействия с ними в виде запросов.

Разберу задачу пошагово — сначала проанализирую требования, затем предложу схему БД, кеш-слой и логику взаимодействия.

Шаг 1. Анализ требований

Ключевые сущности и связи:

- **Товары** — базовый каталог.
- **ГЕО** — географические зоны продаж.
- **Валюты** — привязаны к ГЕО.
- **Цены** — зависят от товара и ГЕО, включают доставку.
- **Лиды** — фиксируются по ГЕО и товару.
- **Коэффициенты** — динамически корректируют цену на основе количества лидов.
- **Рейтинги** — пересчитываются каждые 10 минут.

Важные нюансы:

- Высокая частота обновления коэффициентов (каждые 10 минут).
- Большое количество операций записи лидов.
- Необходимость быстрого чтения актуальных цен для пользователей.

Шаг 2. Схема базы данных

```
-- Товары
CREATE TABLE products (
```

```sql
    product_id BIGINT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ГЕО
CREATE TABLE geo_zones (
    geo_id BIGINT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    currency_code CHAR(3) NOT NULL -- например, USD, EUR
);

-- Базовые цены (с учётом доставки)
CREATE TABLE base_prices (
    product_id BIGINT NOT NULL,
    geo_id BIGINT NOT NULL,
    base_price DECIMAL(15, 2) NOT NULL,
    PRIMARY KEY (product_id, geo_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (geo_id) REFERENCES geo_zones(geo_id)
);

-- Лиды
CREATE TABLE leads (
    lead_id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL,
    geo_id BIGINT NOT NULL,
    lead_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (geo_id) REFERENCES geo_zones(geo_id)
);

-- Коэффициенты и итоговые цены (рассчитываются раз в 10 минут)
CREATE TABLE price_coefficients (
    product_id BIGINT NOT NULL,
    geo_id BIGINT NOT NULL,
    coefficient DECIMAL(5, 4) DEFAULT 1.0,
    calculated_at TIMESTAMP NOT NULL,
    final_price DECIMAL(15, 2) NOT NULL,
    PRIMARY KEY (product_id, geo_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (geo_id) REFERENCES geo_zones(geo_id)
);
```

Шаг 3. Кеш-слой

Используем Redis для:

- кэширования итоговых цен (ключ: `price:{product_id}:{geo_id}`);
- подсчёта текущих лидов в реальном времени (ключ: `leads_count:{product_id}:{geo_id}`).

Стратегия кеширования:

- **Write-through** для базовых цен: запись в БД + обновление кеша.
- **Read-through** для итоговых цен: если в кеше нет — берём из БД, кладем в кеш.
- **TTL** для итоговых цен — 9 минут (чтобы гарантировать пересчет каждые 10 минут).

Шаг 4. Логика взаимодействия и запросы

1. Фиксация лида

Действие: пользователь оставляет заявку.

Операции:

1. Запись в таблицу `leads`.
2. Увеличение счётчика в Redis.

Запросы:

```
-- Запись лида в БД
INSERT INTO leads (product_id, geo_id) VALUES ($1, $2);
```

```
# Увеличение счётчика лидов в Redis
INCR leads_count:{$product_id}:{$geo_id}
```

2. Пересчёт коэффициентов (каждые 10 минут)

Действие: фоновый процесс пересчитывает коэффициенты и итоговые цены.

Логика расчёта:

- Чем больше лидов — тем ниже коэффициент (например, линейная зависимость: $\text{коэффициент} = \frac{1}{1 + k \cdot \text{количество лидов}}$, где k — настраиваемый параметр).
- Итоговая цена: $\text{final_price} = \text{base_price} \times \text{coefficient}$.

Запрос к БД (подсчёт лидов за последние 10 минут):

```
SELECT product_id, geo_id, COUNT(*) AS lead_count
FROM leads
WHERE lead_timestamp >= NOW() - INTERVAL '10 minutes'
GROUP BY product_id, geo_id;
```

Обновление коэффициентов и цен:

```
INSERT INTO price_coefficients (product_id, geo_id, coefficient, calculated_at,
final_price)
VALUES ($1, $2, $3, NOW(), $4)
ON CONFLICT (product_id, geo_id) DO UPDATE
SET coefficient = EXCLUDED.coefficient,
```

```
    calculated_at = EXCLUDED.calculated_at,
    final_price = EXCLUDED.final_price;
```

3. Получение цены для пользователя

Действие: показ цены на сайте/в приложении.

Порядок операций:

1. Проверка кеша Redis по ключу `price:{product_id}:{geo_id}`.
2. Если в кеше нет:
   - запрос к БД: `SELECT final_price FROM price_coefficients WHERE product_id = $1 AND geo_id = $2`;
   - запись результата в кеш с TTL = 9 минут.
3. Возврат цены пользователю.

Пример запроса к Redis:

```
GET price:{$product_id}:{$geo_id}
```

4. Обновление базовой цены

Действие: изменение базовой цены (например, из-за изменения стоимости доставки).

Операции:

1. Обновление `base_prices`.
2. Запуск пересчёта коэффициентов (как в шаге 2).
3. Очистка соответствующего ключа в кеше Redis.

Запрос:

```
UPDATE base_prices
SET base_price = $3
WHERE product_id = $1 AND geo_id = $2;
```

```
DEL price:{$product_id}:{$geo_id}
```

Шаг 5. Обобщённая схема взаимодействия

1. **Пользователь оставляет лид** → запись в `leads` + увеличение счётчика в Redis.
2. **Каждые 10 минут:**
   - подсчёт лидов за период;
   - расчёт коэффициентов;
   - обновление `price_coefficients`;
   - заполнение кеша Redis.

3. **Показ цены:** чтение из кеша (если есть) или из БД + запись в кеш.
4. **Изменение базовой цены:** обновление БД + очистка кеша + пересчёт коэффициентов.

Преимущества решения

- **Производительность:** кеш Redis обеспечивает быстрое чтение цен.
- **Актуальность:** пересчёт каждые 10 минут гарантирует корректность коэффициентов.
- **Масштабируемость:** разделение на базовые цены и коэффициенты позволяет гибко настраивать логику.
- **Надёжность:** все данные сохраняются в реляционной БД, кеш — вторичный слой.